Lab - 1 | 22 January 2026

↳ 6 cells hidden

Lab - 2 | 29 January 2026

↳ 14 cells hidden

Lab - 3 | 05 February 2026

↳ 4 cells hidden

Lab 4 | 12 February 2026

Lab 5 | 19 February 2026

↳ 2 cells hidden

Lab - 6 | 26 February 2026

↳ 7 cells hidden

Lab - 7 | 05 March 2026

↳ 8 cells hidden

Lab 8 | 12 March 2026

Default Code

```
import tensorflow as tf
from tensorflow.keras.datasets import mnist
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Flatten
import numpy as np

# Load dataset
(X_train, y_train), (X_test, y_test) = mnist.load_data()

# Normalize
X_train = X_train / 255.0
X_test = X_test / 255.0

# Flatten
X_train = X_train.reshape(-1, 784)
X_test = X_test.reshape(-1, 784)

# One-hot encoding
y_train = to_categorical(y_train, 10)
y_test = to_categorical(y_test, 10)

# Model
model = Sequential([
    Dense(10, activation='softmax', input_shape=(784,))
])

model.compile(optimizer='adam',
              loss='categorical_crossentropy',
              metrics=['accuracy'])

model.summary()

model.fit(X_train, y_train, epochs=10, batch_size=64)

test_loss, test_acc = model.evaluate(X_test, y_test)
print("Test accuracy:", test_acc)
```

Model: "sequential_12"

Layer (type)	Output Shape	P
dense_15 (Dense)	(None, 10)	

Total params: 7,850 (30.66 KB)

```
 Total params: 7,850 (30.66 KB)
 Trainable params: 7,850 (30.66 KB)
 Non-trainable params: 0 (0.00 B)
Epoch 1/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.7686 - loss:
Epoch 2/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.9078 - loss:
Epoch 3/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.9189 - loss:
Epoch 4/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.9195 - loss:
Epoch 5/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.9218 - loss:
Epoch 6/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - accuracy: 0.9235 - loss:
Epoch 7/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.9267 - loss:
Epoch 8/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 3s 2ms/step - accuracy: 0.9281 - loss:
Epoch 9/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.9292 - loss:
Epoch 10/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.9281 - loss:
313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.9158 - loss:
Test accuracy: 0.9265999794006348
```

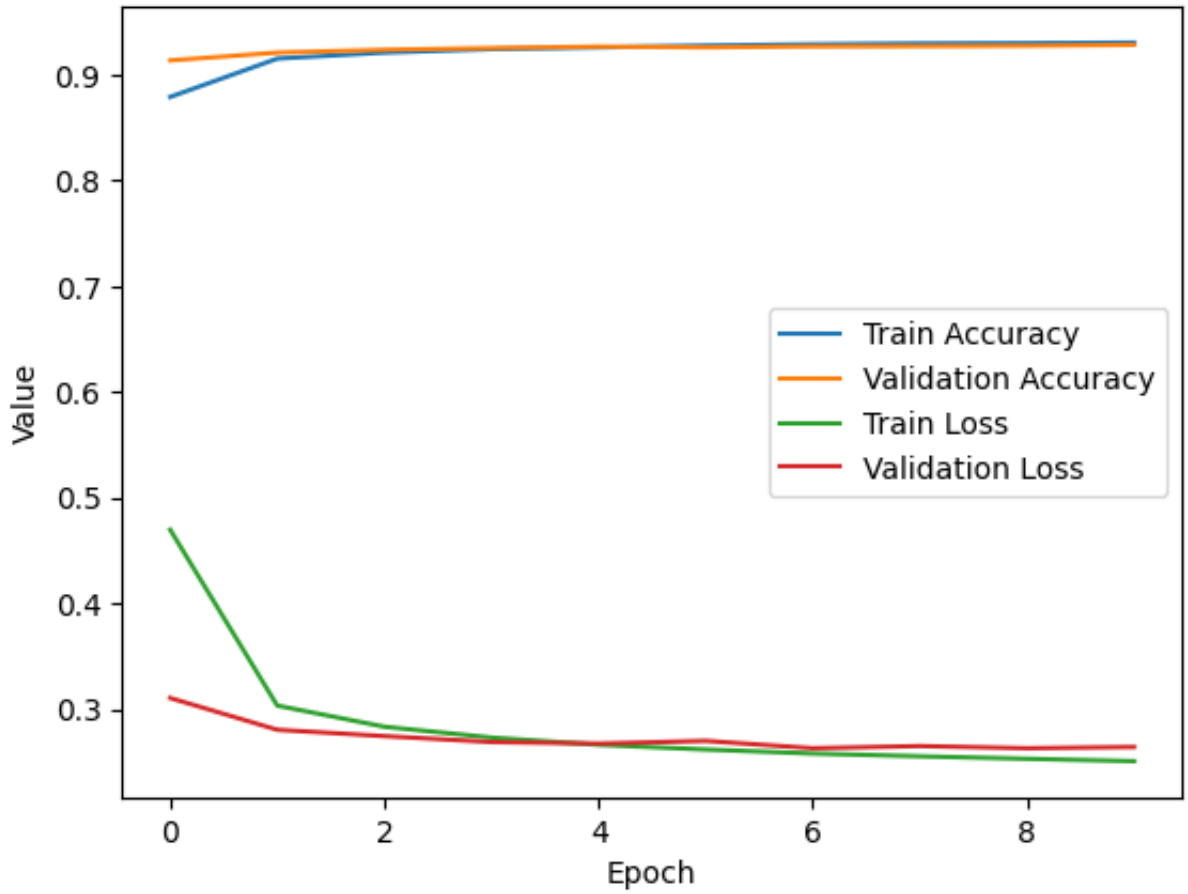
Default + Change the batch size

```
import tensorflow as tf
from tensorflow.keras.datasets import mnist
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
import matplotlib.pyplot as plt

# Load dataset
(X_train, y_train), (X_test, y_test) = mnist.load_data()

# Normalize
X_train = X_train / 255.0
X_test = X_test / 255.0

# Flatten
X_train = X_train.reshape(-1, 784)
X_test = X_test.reshape(-1, 784)

# One hot encoding
y_train = to_categorical(y_train, 10)
y_test = to_categorical(y_test, 10)
```

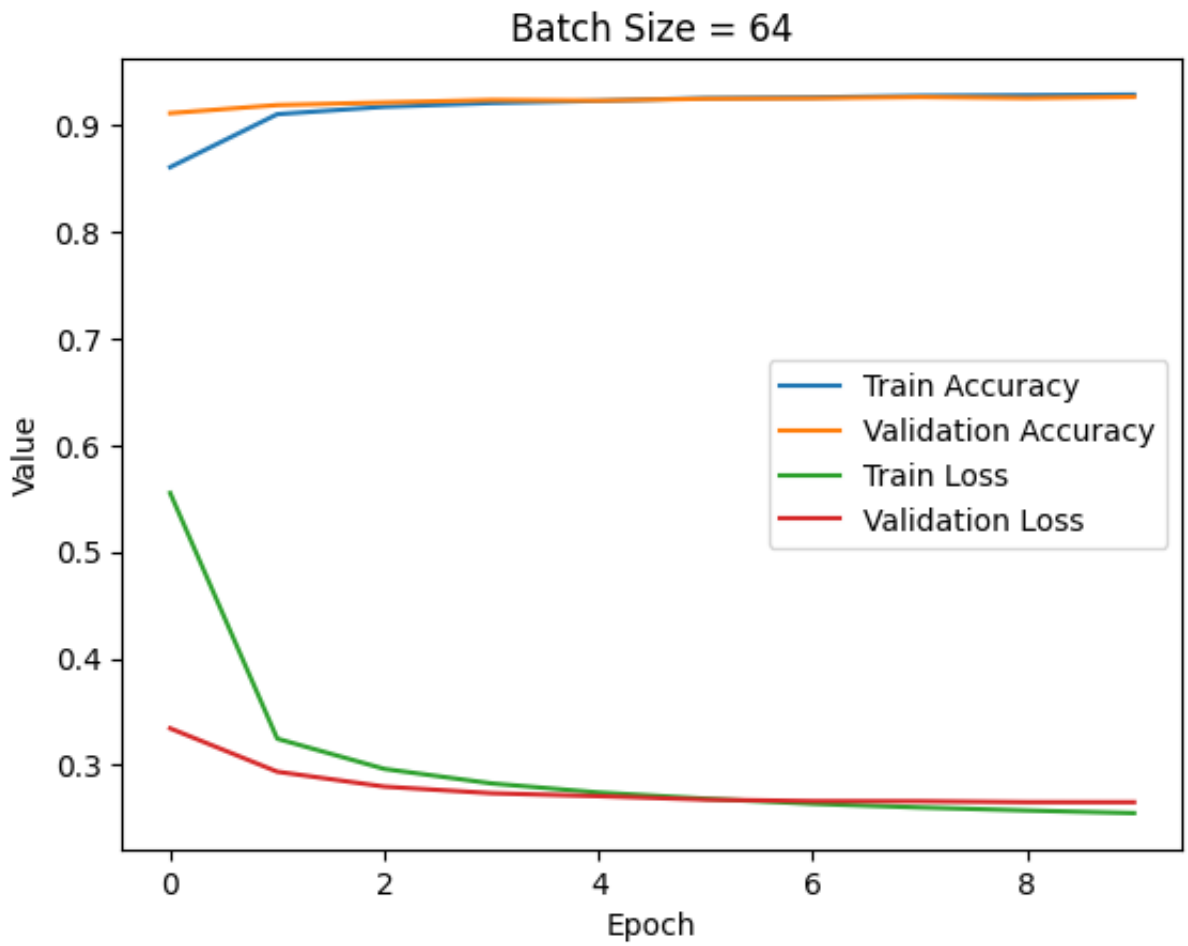
```
# Batch sizes to test
batch_sizes = [32, 64, 128, 256]

for batch in batch_sizes:

    print("Training with batch size:", batch)

    # Model
    model = Sequential([
        Dense(10, activation='softmax', input_shape=(784,))
    ])

    model.compile(
        optimizer='adam',
        loss='categorical_crossentropy',
        metrics=['accuracy']
    )

    # Train model
    history = model.fit(
        X_train,
        y_train,
        epochs=10,
        batch_size=batch,
        validation_data=(X_test, y_test),
        verbose=0
    )

    # Plot graph for this batch
    plt.figure()

    plt.plot(history.history['accuracy'], label='Train Accuracy')
    plt.plot(history.history['val_accuracy'], label='Validation Acc
    plt.plot(history.history['loss'], label='Train Loss')
    plt.plot(history.history['val_loss'], label='Validation Loss')

    plt.title(f'Batch Size = {batch}')
    plt.xlabel("Epoch")
    plt.ylabel("Value")
    plt.legend()

    plt.show()
```

```
Training with batch size: 32
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.
  super().__init__(activity_regularizer=activity_regularizer, **kwar
```

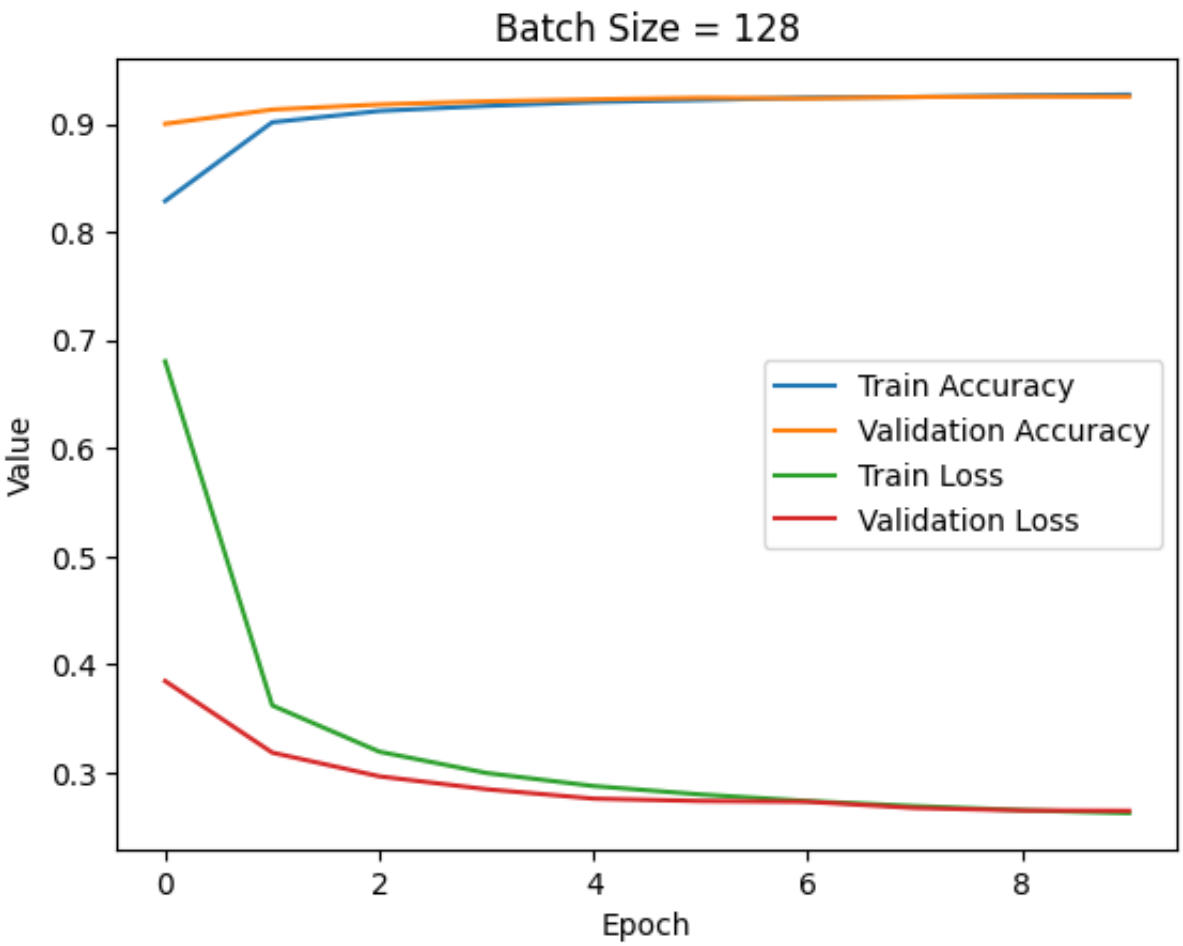
Batch Size = 32



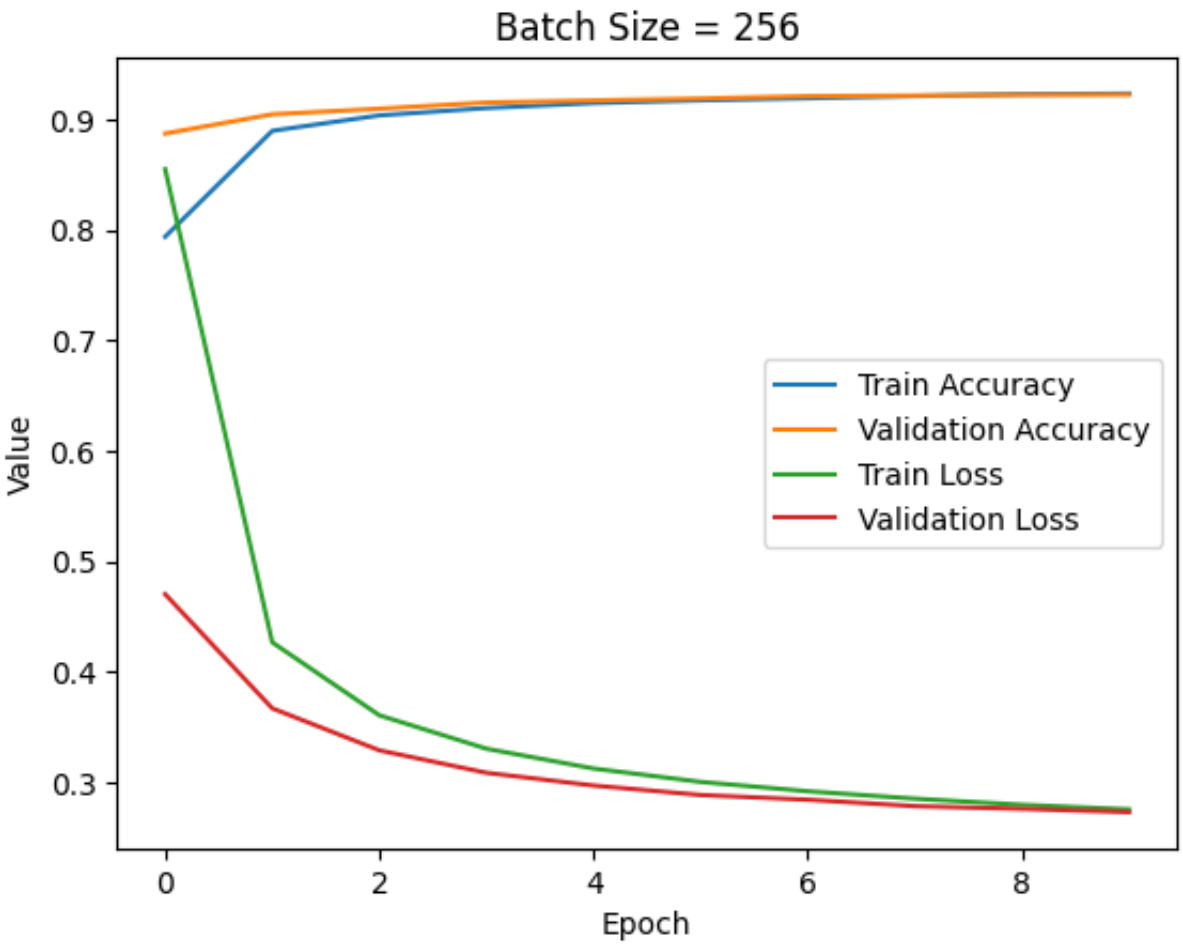

```
Training with batch size: 64
```




```
Training with batch size: 128
```




```
Training with batch size: 256
```

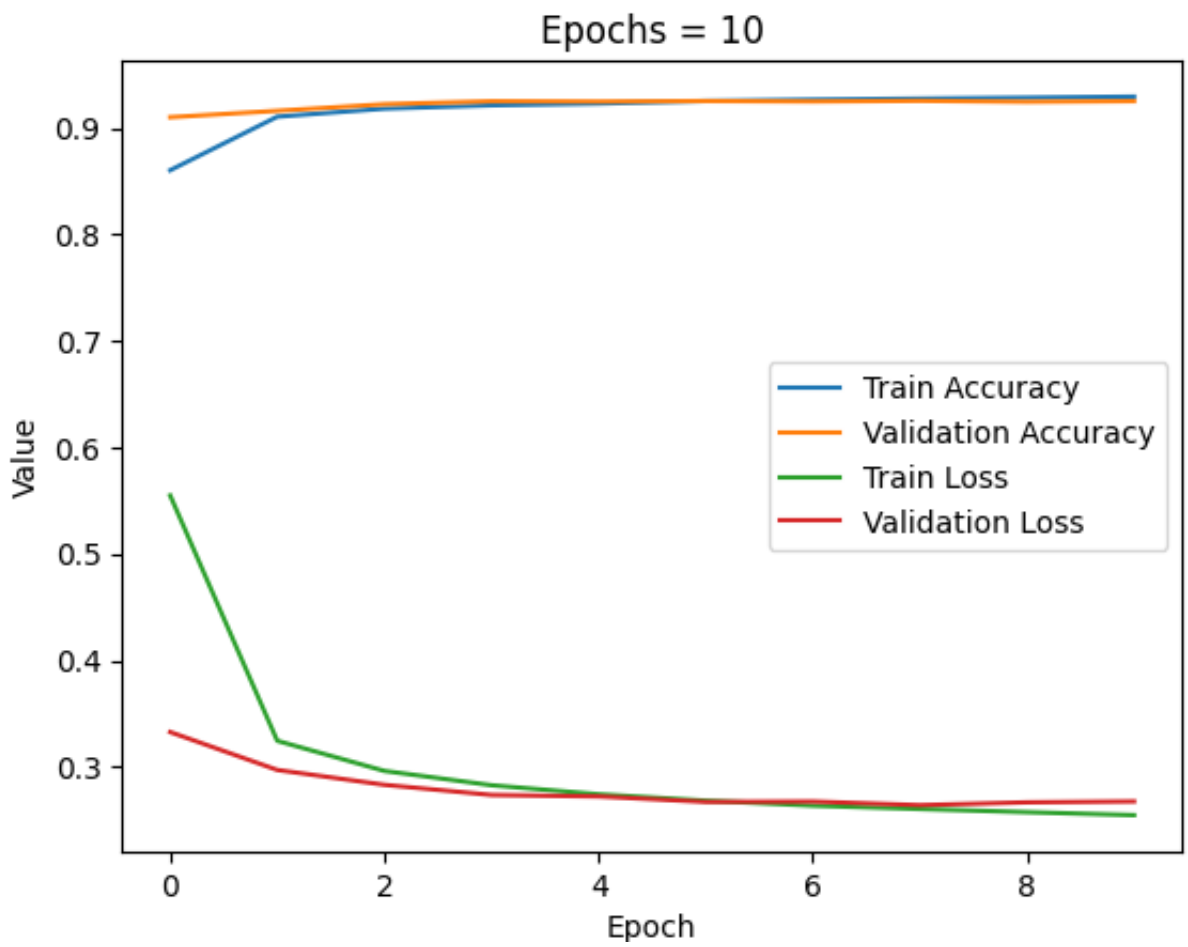


Default + Change no. of Epochs

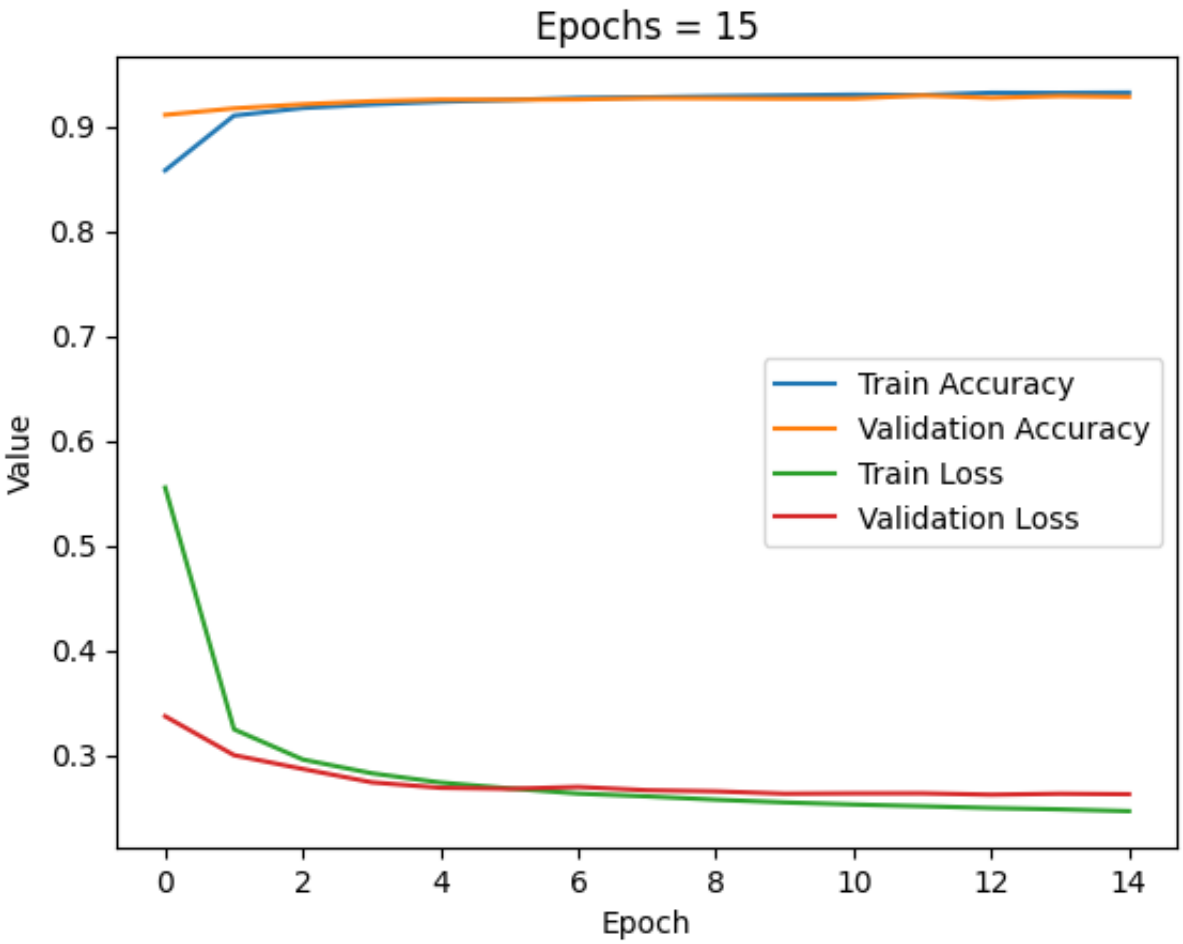
```
import tensorflow as tf
from tensorflow.keras.datasets import mnist
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
import matplotlib.pyplot as plt

# Load dataset
(X_train, y_train), (X_test, y_test) = mnist.load_data()

# Normalize
X_train = X_train / 255.0
X_test = X_test / 255.0

# Flatten images
X_train = X_train.reshape(-1, 784)
X_test = X_test.reshape(-1, 784)

# One-hot encoding
y_train = to_categorical(y_train, 10)
y_test = to_categorical(y_test, 10)

# Epoch values to test
epoch_values = [10, 15, 20]

for ep in epoch_values:

    print("Training with epochs:", ep)

    # Model
    model = Sequential([
        Dense(10, activation='softmax', input_shape=(784,))
    ])

    model.compile(
        optimizer='adam',
        loss='categorical_crossentropy',
        metrics=['accuracy']
    )

    # Train model
```

```
history = model.fit(
    X_train,
    y_train,
    epochs=ep,
    batch_size=64,
    validation_data=(X_test, y_test),
    verbose=0
)

# Plot graph
plt.figure()

plt.plot(history.history['accuracy'], label='Train Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Acc
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')

plt.title(f'Epochs = {ep}')
plt.xlabel("Epoch")
plt.ylabel("Value")
plt.legend()

plt.show()
```

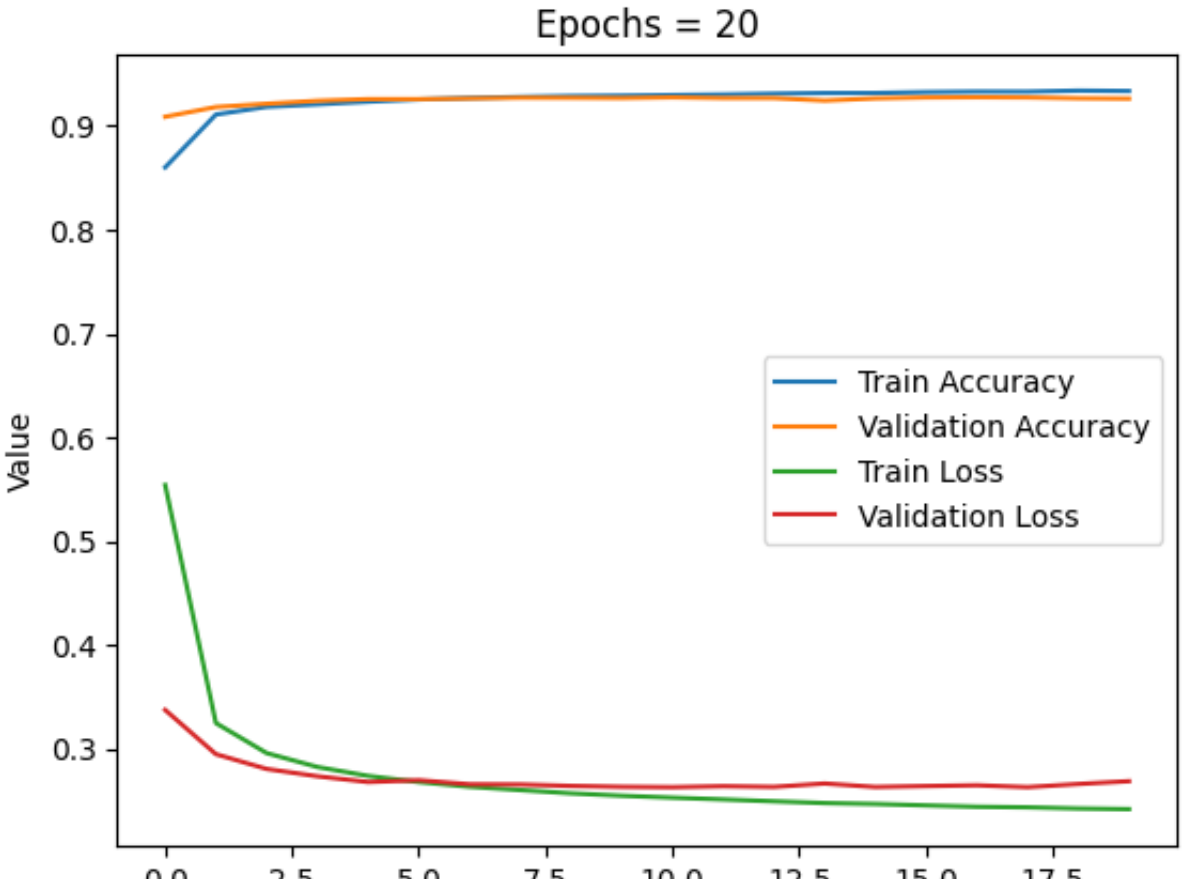
```
Training with epochs: 10
```

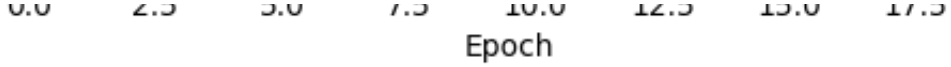


```
Training with epochs: 15
```




```
Training with epochs: 20
```





Default + Accuracy and Loss Curves

```
import tensorflow as tf
from tensorflow.keras.datasets import mnist
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
import matplotlib.pyplot as plt
import numpy as np

# 1. Load MNIST dataset
(X_train, y_train), (X_test, y_test) = mnist.load_data()

# 2. Normalize pixel values
X_train = X_train / 255.0
X_test = X_test / 255.0

# 3. Flatten images (28x28 → 784)
X_train = X_train.reshape(-1, 784)
X_test = X_test.reshape(-1, 784)

# 4. One-hot encoding
y_train = to_categorical(y_train, 10)
y_test = to_categorical(y_test, 10)

# 5. Build model
model = Sequential([
    Dense(10, activation='softmax', input_shape=(784,))
])

# 6. Compile model
model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

# 7. Model summary
model.summary()

# 8. Train model
history = model.fit(
```

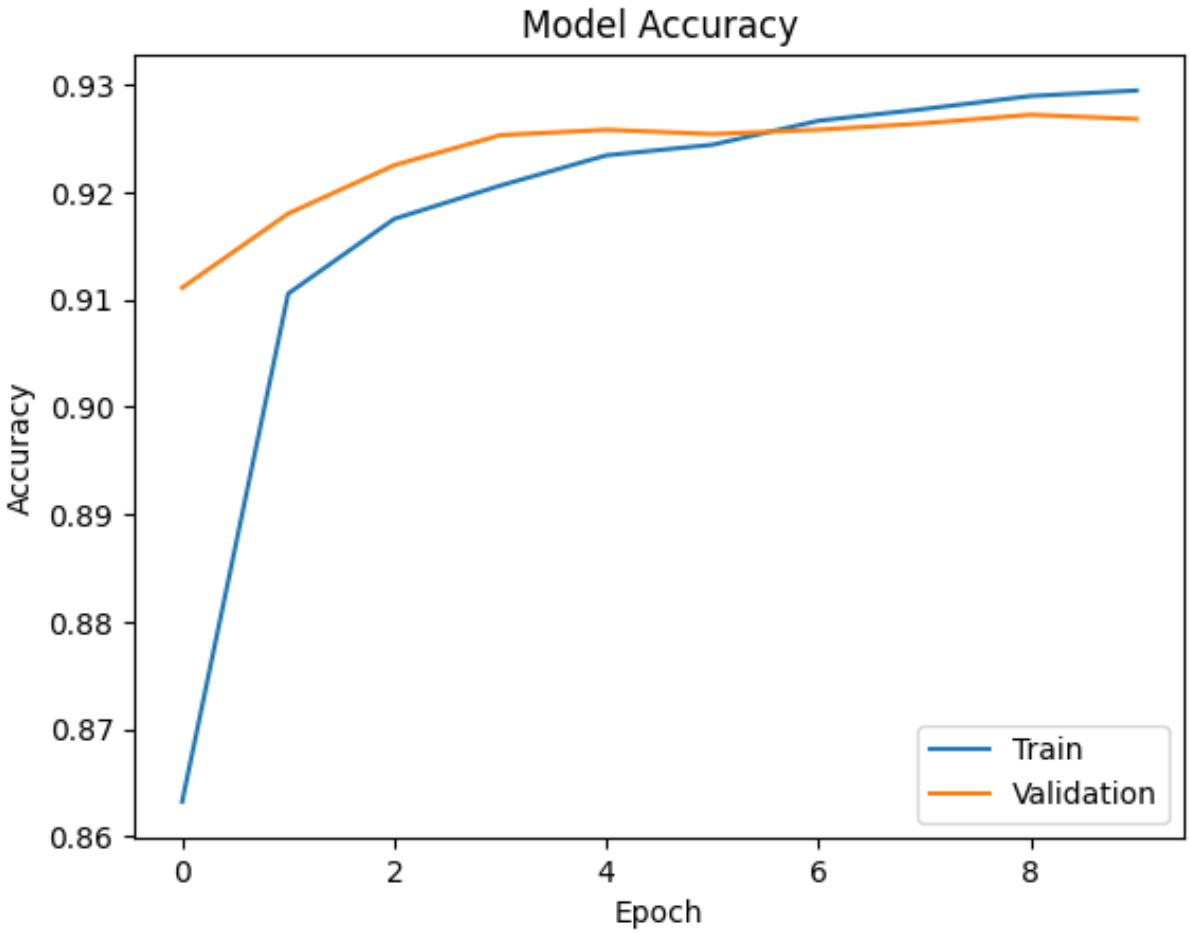
```
    X_train,
    y_train,
    epochs=10,
    batch_size=64,
    validation_data=(X_test, y_test)
)

# 9. Evaluate model
test_loss, test_acc = model.evaluate(X_test, y_test)
print("Test accuracy:", test_acc)

# 10. Plot Accuracy Curve
plt.figure()
plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])
plt.title("Model Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend(["Train", "Validation"])
plt.show()

# 11. Plot Loss Curve
plt.figure()
plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])
plt.title("Model Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend(["Train", "Validation"])
plt.show()
```

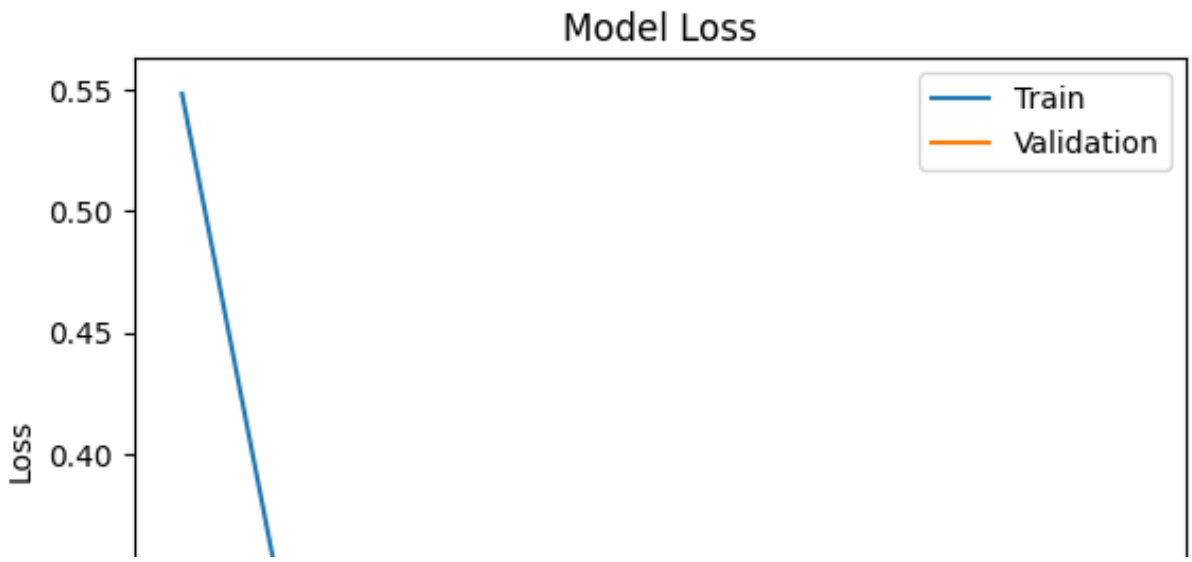
```
Model: "sequential_2"
```

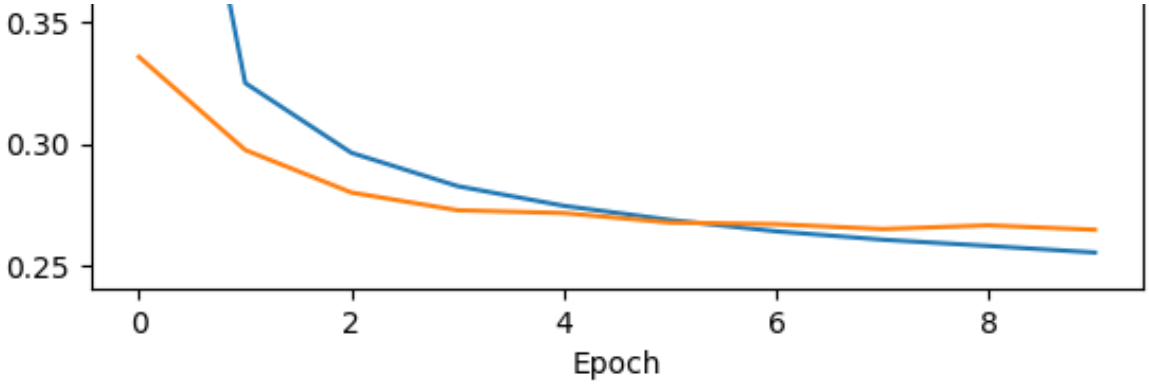
Layer (type)	Output Shape	P
dense_2 (Dense)	(None, 10)	

```
 Total params: 7,850 (30.66 KB)
 Trainable params: 7,850 (30.66 KB)
 Non-trainable params: 0 (0.00 B)
Epoch 1/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 4s 3ms/step - accuracy: 0.7866 - loss:
Epoch 2/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - accuracy: 0.9088 - loss:
Epoch 3/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.9173 - loss:
Epoch 4/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 3s 2ms/step - accuracy: 0.9208 - loss:
Epoch 5/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.9236 - loss:
```

```
Epoch 6/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - accuracy: 0.9245 - loss:
Epoch 7/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - accuracy: 0.9267 - loss:
Epoch 8/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.9274 - loss:
Epoch 9/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.9271 - loss:
Epoch 10/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.9292 - loss:
313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.9163 - loss:
Test accuracy: 0.926800012588501
```







ADD Next Layer using Reloc

32 hidden layers

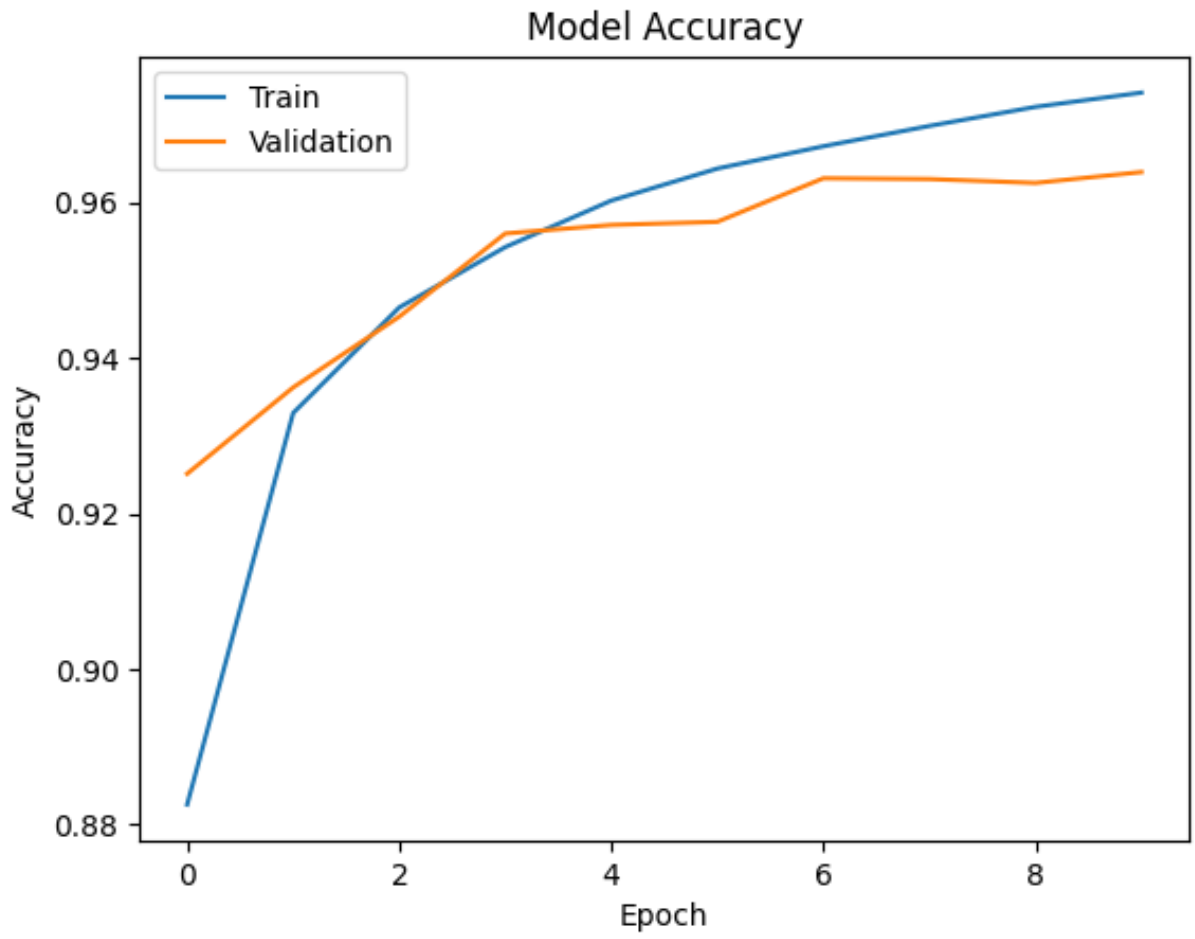
```
import tensorflow as tf
from tensorflow.keras.datasets import mnist
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
import matplotlib.pyplot as plt
import numpy as np

# 1. Load MNIST dataset
(X_train, y_train), (X_test, y_test) = mnist.load_data()

# 2. Normalize pixel values
X_train = X_train / 255.0
X_test = X_test / 255.0

# 3. Flatten images (28x28 → 784)
X_train = X_train.reshape(-1, 784)
X_test = X_test.reshape(-1, 784)

# 4. One-hot encoding
y_train = to_categorical(y_train, 10)
y_test = to_categorical(y_test, 10)

# 5. Build model (Added hidden layer with 32 neurons)
model = Sequential([
    Dense(32, activation='relu', input_shape=(784,)),   # Hidden la
    Dense(10, activation='softmax')                     # Output la
])

# 6. Compile model
```

```
model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

# 7. Model summary
model.summary()

# 8. Train model
history = model.fit(
    X_train,
    y_train,
    epochs=10,
    batch_size=64,
    validation_data=(X_test, y_test)
)

# 9. Evaluate model
test_loss, test_acc = model.evaluate(X_test, y_test)
print("Test accuracy:", test_acc)

# 10. Plot Accuracy Curve
plt.figure()
plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])
plt.title("Model Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend(["Train", "Validation"])
plt.show()

# 11. Plot Loss Curve
plt.figure()
plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])
plt.title("Model Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend(["Train", "Validation"])
plt.show()
```

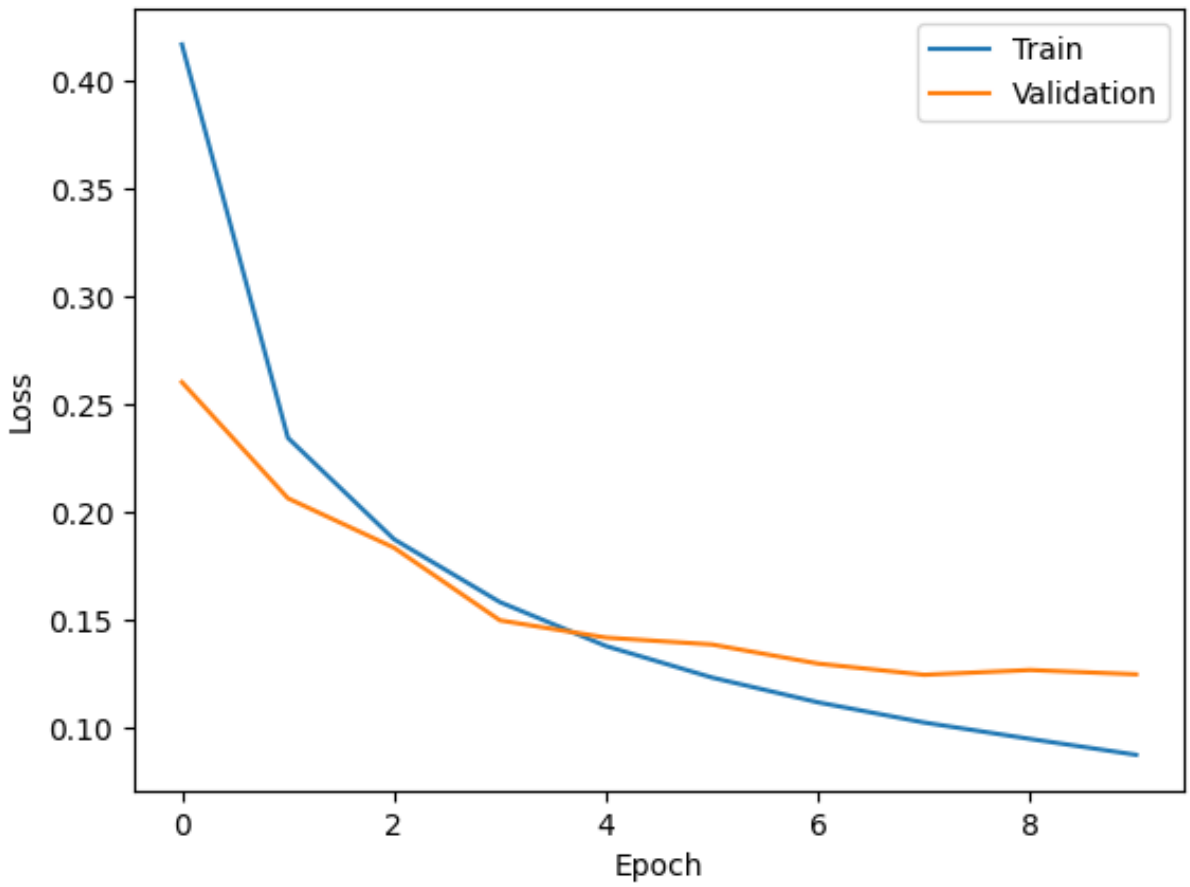
Model: "sequential_10"

Layer (type)	Output Shape	P
dense_10 (Dense)	(None, 32)	
dense_11 (Dense)	(None, 10)	

```
 Total params: 25,450 (99.41 KB)
 Trainable params: 25,450 (99.41 KB)
 Non-trainable params: 0 (0.00 B)
Epoch 1/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 4s 3ms/step - accuracy: 0.8055 - loss:
Epoch 2/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 9s 7ms/step - accuracy: 0.9300 - loss:
Epoch 3/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 8s 5ms/step - accuracy: 0.9443 - loss:
Epoch 4/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 6s 6ms/step - accuracy: 0.9545 - loss:
Epoch 5/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 7s 3ms/step - accuracy: 0.9598 - loss:
Epoch 6/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - accuracy: 0.9645 - loss:
Epoch 7/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.9691 - loss:
Epoch 8/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 4s 4ms/step - accuracy: 0.9702 - loss:
Epoch 9/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 8s 9ms/step - accuracy: 0.9727 - loss:
Epoch 10/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.9756 - loss:
313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.9579 - loss:
Test accuracy: 0.9639000296592712
```



Model Loss



64 Hidden Layers

```
import tensorflow as tf
from tensorflow.keras.datasets import mnist
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
import matplotlib.pyplot as plt
import numpy as np

# 1. Load MNIST dataset
(X_train, y_train), (X_test, y_test) = mnist.load_data()

# 2. Normalize pixel values
X_train = X_train / 255.0
X_test = X_test / 255.0

# 3. Flatten images (28x28 → 784)
X_train = X_train.reshape(-1, 784)
X_test = X_test.reshape(-1, 784)

# 4. One-hot encoding
y_train = to_categorical(y_train, 10)
```

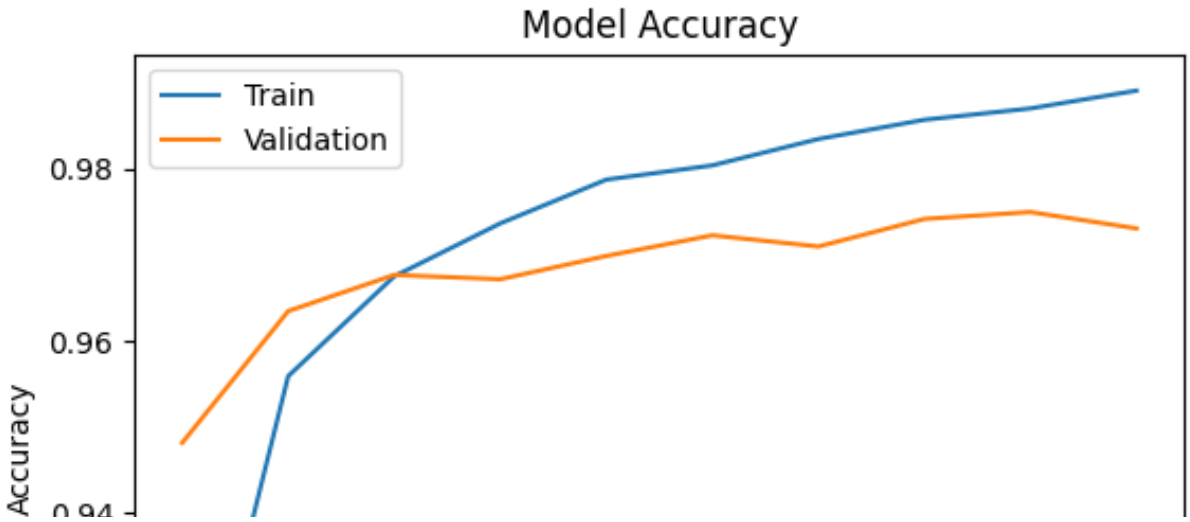
```
y_test = to_categorical(y_test, 10)

# 5. Build model (Added 64 and 32 neuron layers)
model = Sequential([
    Dense(64, activation='relu', input_shape=(784,)),  # First hidd
    Dense(32, activation='relu'),                      # Second hid
    Dense(10, activation='softmax')                    # Output lay
])

# 6. Compile model
model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

# 7. Model summary
model.summary()

# 8. Train model
history = model.fit(
    X_train,
    y_train,
    epochs=10,
    batch_size=64,
    validation_data=(X_test, y_test)
)

# 9. Evaluate model
test_loss, test_acc = model.evaluate(X_test, y_test)
print("Test accuracy:", test_acc)

# 10. Plot Accuracy Curve
plt.figure()
plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])
plt.title("Model Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend(["Train", "Validation"])
plt.show()

# 11. Plot Loss Curve
plt.figure()
plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])
plt.title("Model Loss")
plt.xlabel("Epoch")
```

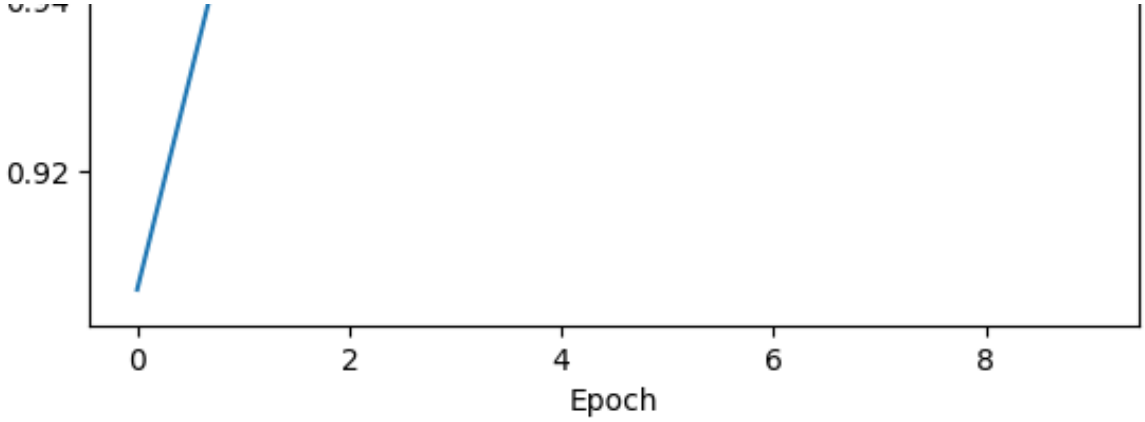
```
plt.ylabel("Loss")
plt.legend(["Train", "Validation"])
plt.show()
```

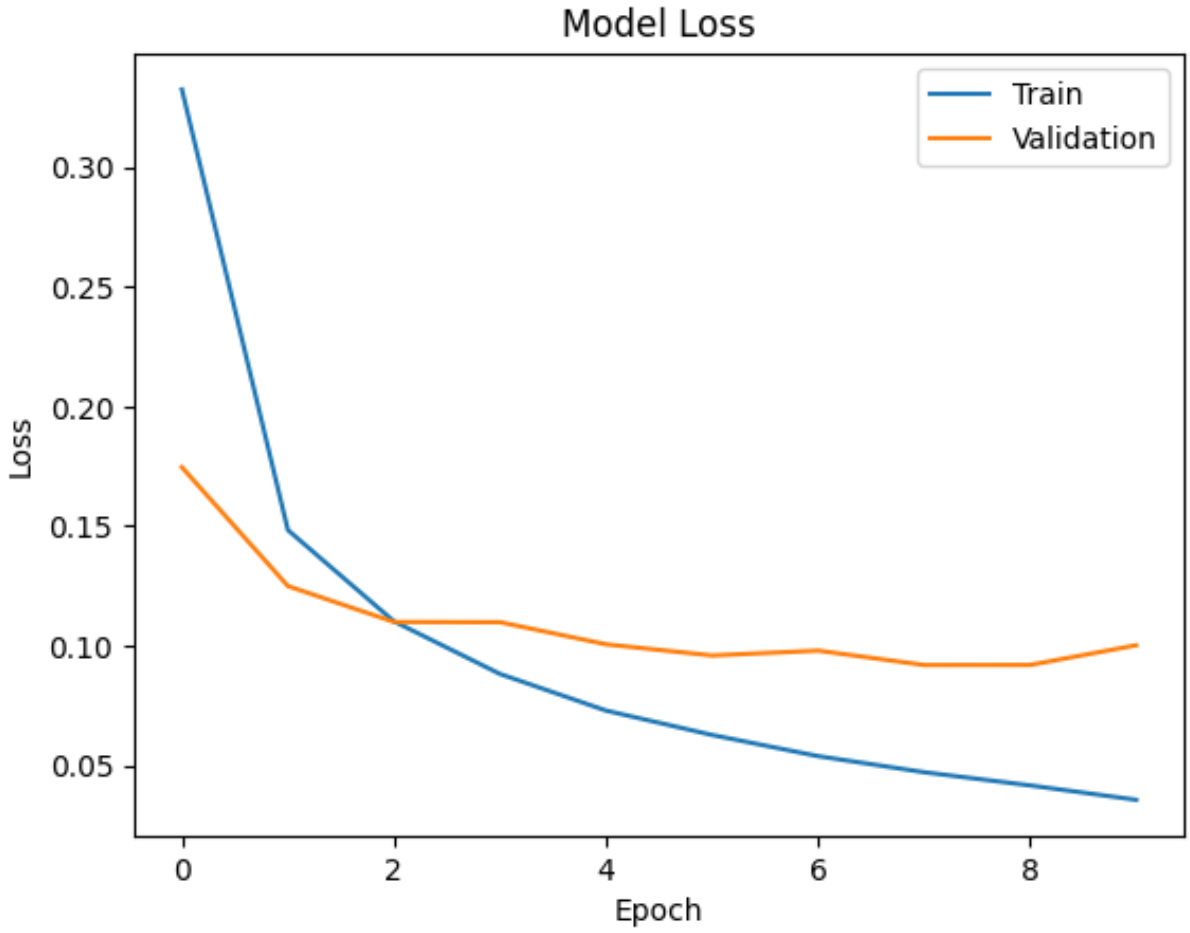
Model: "sequential_11"

Layer (type)	Output Shape	P
dense_12 (Dense)	(None, 64)	
dense_13 (Dense)	(None, 32)	
dense_14 (Dense)	(None, 10)	

```
 Total params: 52,650 (205.66 KB)
 Trainable params: 52,650 (205.66 KB)
 Non-trainable params: 0 (0.00 B)
Epoch 1/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 6s 5ms/step - accuracy: 0.8321 - loss:
Epoch 2/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 4s 3ms/step - accuracy: 0.9543 - loss:
Epoch 3/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - accuracy: 0.9657 - loss:
Epoch 4/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 4s 4ms/step - accuracy: 0.9732 - loss:
Epoch 5/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - accuracy: 0.9789 - loss:
Epoch 6/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 3s 4ms/step - accuracy: 0.9807 - loss:
Epoch 7/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 5s 4ms/step - accuracy: 0.9845 - loss:
Epoch 8/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 4s 4ms/step - accuracy: 0.9861 - loss:
Epoch 9/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - accuracy: 0.9878 - loss:
Epoch 10/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - accuracy: 0.9899 - loss:
313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.9695 - loss:
Test accuracy: 0.9729999899864197
```







OCR Detection

```
import tensorflow as tf
from tensorflow.keras.datasets import mnist
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
import matplotlib.pyplot as plt
import numpy as np
from PIL import Image
```

```
from google.colab import files

# 1. Load MNIST dataset
(X_train, y_train), (X_test, y_test) = mnist.load_data()

# 2. Normalize pixel values
X_train = X_train / 255.0
X_test = X_test / 255.0

# 3. Flatten images (28x28 → 784)
X_train = X_train.reshape(-1, 784)
X_test = X_test.reshape(-1, 784)

# 4. One-hot encoding
y_train = to_categorical(y_train, 10)
y_test = to_categorical(y_test, 10)

# 5. Build neural network
model = Sequential([
    Dense(128, activation='relu', input_shape=(784,)),
    Dense(64, activation='relu'),
    Dense(32, activation='relu'),
    Dense(10, activation='softmax')
])

# 6. Compile model
model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

# 7. Train model
history = model.fit(
    X_train,
    y_train,
    epochs=10,
    batch_size=64,
    validation_data=(X_test, y_test)
)

# 8. Evaluate model
test_loss, test_acc = model.evaluate(X_test, y_test)
print("Test Accuracy:", test_acc)

# 9. Upload handwritten image
print("Upload your handwritten digit image")
uploaded = files.upload()
```

```
# 10. Load image
filename = list(uploaded.keys())[0]
img = Image.open(filename).convert("L")

# 11. Resize to MNIST format
img = img.resize((28,28))

# 12. Convert to numpy
img_array = np.array(img)

# 13. Invert colors (MNIST uses white digit on black background)
img_array = 255 - img_array

# 14. Normalize
img_array = img_array / 255.0

# 15. Show processed image
plt.imshow(img_array, cmap='gray')
plt.title("Processed Input Image")
plt.axis("off")
plt.show()

# 16. Flatten image
img_array = img_array.reshape(1,784)

# 17. Predict digit
prediction = model.predict(img_array)

digit = np.argmax(prediction)

print("Prediction probabilities:", prediction)
print("Predicted Digit:", digit)
```

```
Epoch 1/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 13s 11ms/step - accuracy: 0.8260 - loss
Epoch 2/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 4s 4ms/step - accuracy: 0.9655 - loss:
Epoch 3/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 5s 6ms/step - accuracy: 0.9759 - loss:
Epoch 4/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 4s 4ms/step - accuracy: 0.9825 - loss:
Epoch 5/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 5s 5ms/step - accuracy: 0.9847 - loss:
Epoch 6/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 5s 5ms/step - accuracy: 0.9885 - loss:
Epoch 7/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 4s 4ms/step - accuracy: 0.9910 - loss:
Epoch 8/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 5s 5ms/step - accuracy: 0.9919 - loss:
```

```
Epoch 9/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 4s 5ms/step - accuracy: 0.9930 - loss:
Epoch 10/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 4s 4ms/step - accuracy: 0.9939 - loss:
313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.9729 - loss:
Test Accuracy: 0.9753999710083008
Upload your handwritten digit image
```

Choose Files **5** ChatGPT Im...00 AM.png

ChatGPT Image Mar 12, 2026 at 11_11_00 AM.png(image/png) - 193958 bytes, last modified: n/a - 100% done

```
Saving ChatGPT Image Mar 12, 2026 at 11_11_00 AM.png to ChatGPT Imag
```

Processed Input Image

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 87ms/step
Prediction probabilities: [[1.7705830e-02 3.4021164e-06 1.1809054e-0
  3.7447333e-01 3.7311471e-01 2.7708886e-03 3.7349924e-05 2.3076081e
Predicted Digit: 5
```